

Week 6 Technical Report - Dynamic Programming II

Project repository (complete source, tests, benchmarks and results):

<https://github.com/RDeibel2025/CTX-Advanced-Algorithms>

Dynamic programming II: `src/dp_advanced/` · demonstration: `examples/week6_demo.py`

Benchmark: `benchmarks/week6_dp_advanced_benchmark.py`

Robert Deibel · CSC 5300 Advanced Algorithms · Concordia University Texas · Fall 2026



Complete project repository

<https://github.com/RDeibel2025/CTX-Advanced-Algorithms>

1. Executive Summary

The four algorithms fall into two pairs. Knapsack and Floyd-Warshall are written with one more table dimension than they need, and dropping it cut peak memory by up to 152.7x and 26.8x without changing how runtime grows. Matrix chain multiplication and the bitmask TSP have state spaces that cannot be shrunk, and their cost follows its size: the matrix-chain table was 11,992x faster than plain recursion at 16 matrices, and Held-Karp 2,280x faster than brute force at 12 cities.

2. Methodology

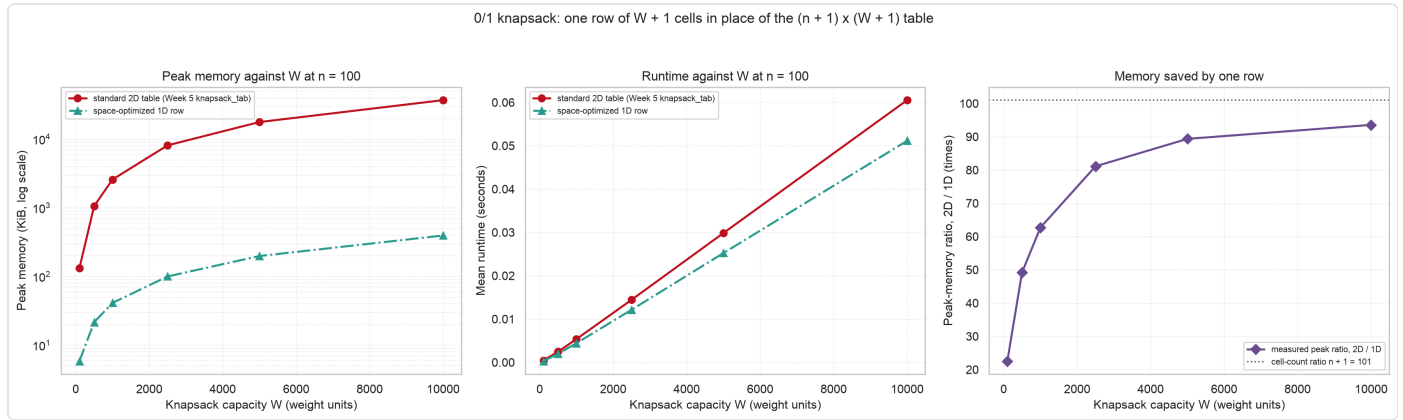
Machine and timing. Apple M2 Max, 12 cores, 64 GB RAM; macOS 14.5 (arm64); CPython 3.12.4. Timing uses `time.perf_counter` through `src/utils/timer.py`, with garbage collection paused per run. Each measurement starts with a pilot run that sets its repetitions: 5 runs after 2 warm-ups below 50 ms, 5 after 1 below 0.5 s, and 3 with the pilot as the warm-up above that. Peak memory is the tracemalloc peak, taken in a separate pass.

Inputs. Seed 42 throughout. Knapsack: weights 1-50, values 1-100, $W = 100$ to 10,000 at 100 items and 25 to 400 items at $W = 1,000$. Matrix chains: dimensions 5 to 100; recursion at 4 to 16 matrices, memoized and bottom-up at 4 to 200. Floyd-Warshall: connected directed graphs from the Week 4 generator, weights 1-100, densities 0.05 and 0.5; 25 to 500 vertices at density 0.5, and against Dijkstra from every source at 50, 100 and 200 vertices at both densities, with the two distance matrices checked equal first. TSP: complete directed matrices, weights 1-100; the bitmask DP at 4 to 18 cities and brute force at 4 to 12, with costs checked equal wherever both ran.

Reductions. Every required size was measured and nothing was projected. Five of the 90 rows ran 3 times rather than 5 because a single run took over half a second: plain recursion at 16 matrices, Floyd-Warshall at 500 vertices, brute force at 11 and 12 cities, and the bitmask DP at 18. Floyd-Warshall at 500 and brute force at 12, both over 2 seconds a run, were timed but not traced for memory, since tracemalloc slows execution by up to an order of magnitude. Every figure below is recomputed from the CSV by `tools/week6_facts.py`.

3. Results

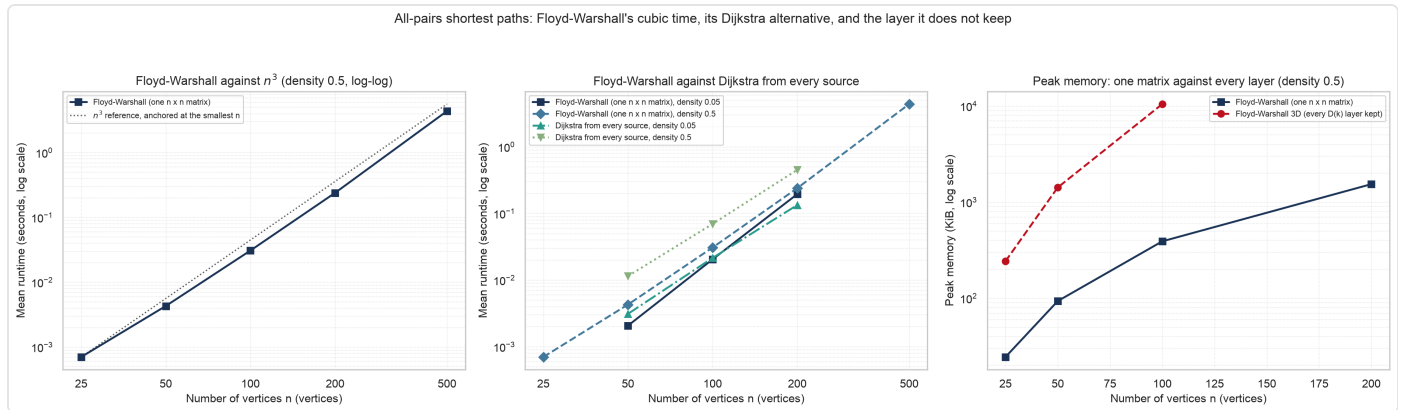
3.1 Standard against space-optimized knapsack



At 100 items, raising W from 100 to 10,000 took the Week 5 table from 130.7 to 36,723.4 KiB and the one-row version from 5.8 to 392.5 KiB. At $W = 1,000$, the one-row peak stayed between 39.2 and 45.7 KiB from 25 to 400 items while the table grew from 772.1 to 6,978.4 KiB: $O(W)$ against $O(n \cdot W)$, measured. Runtime order did not change. From $W = 1,000$ to 10,000 the two grew 11.3x and 11.6x, and the one-row version was 1.17x to 1.66x faster everywhere, since it allocates no new rows.

Theory puts the memory ratio at $n + 1 = 101$; it approached that from below, from 22.5x at $W = 100$ to 93.6x at $W = 10,000$. At small W fixed overhead dominates the one-row peak, and across item counts the table's cost per cell fell from 30.4 bytes at 25 items to 17.8 at 400, because unchanged cells in later rows point at integer objects an earlier row already created.

3.2 Floyd-Warshall against Dijkstra

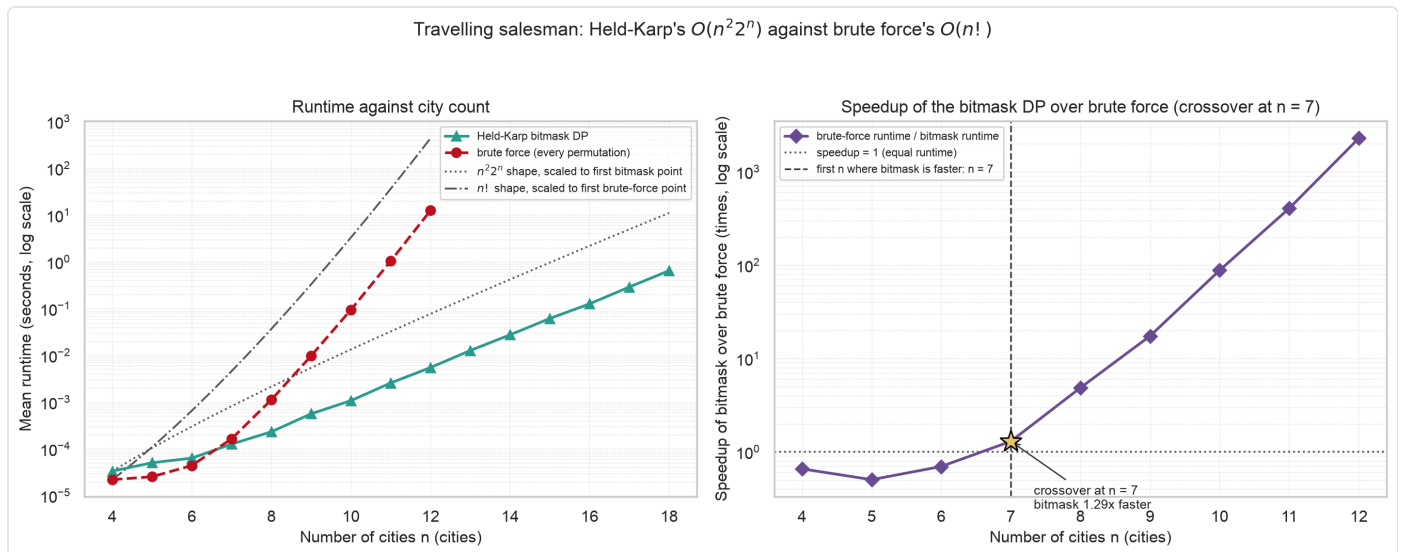
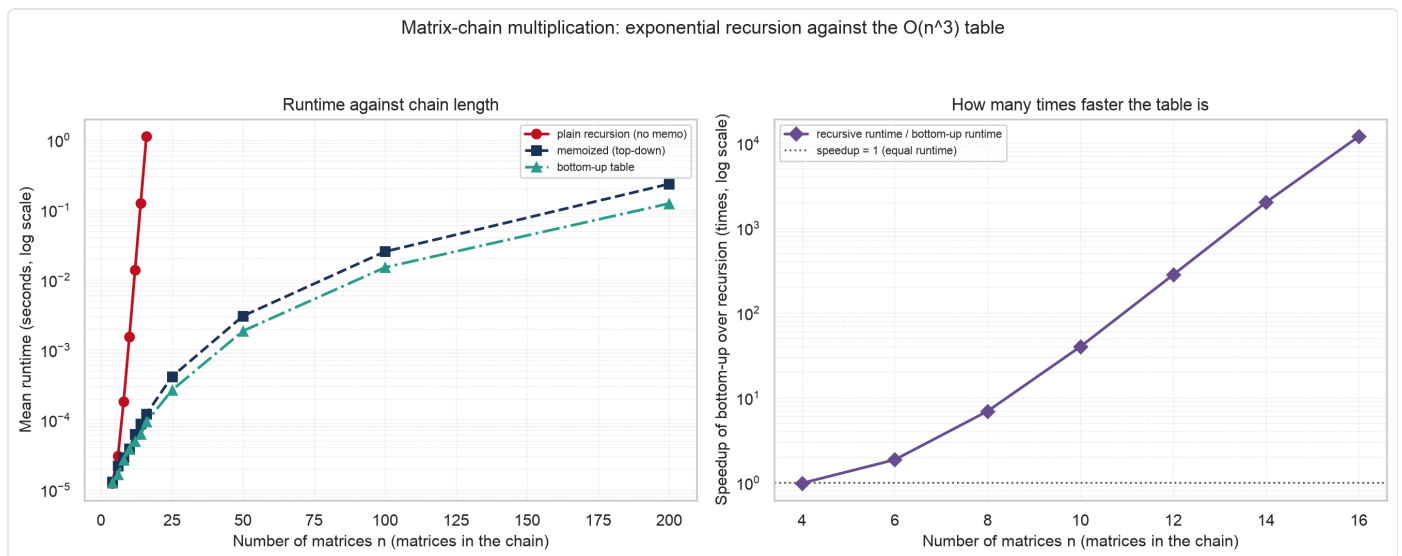


At density 0.5, Floyd-Warshall took 0.7 ms at 25 vertices, 30.7 ms at 100, 238.5 ms at 200 and 4,358.1 ms at 500, a log-log slope of 2.917 against the 3 of $O(V^3)$. From 100 to 200 vertices time rose 7.77x against a predicted 8. From 200 to 500 it rose 18.27x against 15.62x: at 500 the 250,000-entry matrix no longer fits the processor's caches, at 34.9 ns per inner step.

The textbook form that keeps every layer used 9.9x the memory at 25 vertices and 26.8x at 100 (10,445.7 against 389.9 KiB). The ratio grows with n but stays below $n + 1$, for the same shared-object reason as the knapsack table. Both forms are $O(V^3)$: from 50 to 100 vertices the three-dimensional version grew 7.92x and the two-dimensional 7.17x.

Against Dijkstra from every source, density decided the winner, though later than the operation counts suggest. On dense graphs Floyd-Warshall was faster at every size, 2.67x at 50 vertices and 1.88x at 200. On sparse graphs it was still faster at 50 (1.50x) and 100 (1.05x), and Dijkstra won only at 200, 133.1 ms against 194.1. Repeated Dijkstra does fewer basic steps on a sparse graph, but each is a pure-Python operation on the Week 3 heap, so its advantage needed 200 vertices to appear. On the CLRS example with negative edges, Floyd-Warshall returned the textbook matrix and Week 4's Dijkstra refused to run.

3.3 Matrix chain and TSP scalability



Plain matrix-chain recursion went from 0.012 ms at 4 matrices to 1,119.9 ms at 16, and from 10 matrices on each added matrix multiplied its time by 3.00, matching its exact call count of $3^{(n-1)}$. The bottom-up table took 0.093 ms at 16 matrices, 11,992x faster, and grew 8.18x from 100 to 200 matrices against the 8 of $O(n^3)$.

At 4 matrices the two were equal, because the table only pays once there is repeated work to save. Memoization ran 1.90x slower than bottom-up at 200 matrices but used less memory, 947.9 against 1,276.4 KiB, since bottom-up keeps both the cost and split tables.

Brute-force TSP grew 12.02x from 11 to 12 cities against the 11x of $(n-1)!$, reaching 12,502.9 ms. The bitmask DP grew 2.23x from 17 to 18 cities against 2.24x for $n^2 2^n$ and took 647.0 ms at 18. Brute force was faster up to 6 cities, where 120 orderings cost less than building the table; from 7 cities the bitmask DP led, by 2,280x at 12. Its memory reached 73,774.2 KiB at 18 cities, 2.118x the figure at 17, exactly the $n * 2^n$ prediction.

4. Discussion

Tables with a dimension they do not need. Knapsack and Floyd-Warshall are both written with an extra dimension, the item index and the intermediate vertex k , and both drop it by overwriting a smaller table in place, for different reasons. In the knapsack, row i reads row $i-1$ only at capacities c and $c - w_i$, so scanning c downward leaves $dp[c - w_i]$ untouched when $dp[c]$ reads it. Scanning upward takes an item again: with weights 2 and 3, values 3 and 4 and capacity 6, the downward scan returns 7 and the upward scan 9, the unbounded answer (Amakobe §6.7). In Floyd-Warshall the direction does not matter, because with no negative cycle $dist[k][k]$ is 0, so row k and column k cannot change during iteration k (CLRS §23.2). In both, time was unchanged and memory fell.

State spaces that cannot be shrunk. Matrix chain keeps every interval and TSP every subset to the end, because a longer interval or larger subset may need any of them. Matrix chain has about $n^2/2$ intervals with up to n split points each, which is polynomial (CLRS §14.2; Amakobe §6.5). Held-Karp has 2^n subsets times n end cities with n transitions each, exponential but far smaller than $n!$ (Held & Karp, 1962; Amakobe §6.8): at 12 cities, 589,824 transitions against 39,916,800 orderings.

When to prioritize space. The one-row knapsack was faster as well as smaller, so for the optimum alone the table is not worth keeping. Its cost is reconstruction, since one row does not record which choice produced each cell. The module recovers the items with $n * W$ bits rather than $n * W$ integers, which reduces that cost without removing it; Hirschberg (1975) gives the general linear-space method. The bitmask DP has no such option: it needed 72 MB at 18 cities, doubling with each city, so memory limits it before time does. Its readability cost is that one loop's direction carries the whole correctness argument and is easy to reverse unnoticed, which is why the upward scan is kept as a test.

Applications. Query optimizers order joins the way matrix chain multiplication orders products, route planning is TSP, and genome alignment uses LCS-style tables with Hirschberg's linear-space method for long sequences.

5. Conclusion

Week 5 turned exponential recursion into polynomial tables. This week adds two questions to ask of any such table: which dimension it can drop, and how large its state space must be. The first decides memory. The second decides feasibility: the matrix-chain table is cheap at 200 matrices, while Held-Karp needs 72 MB at 18 cities and doubles with each city after that. The trade-off left open is the knapsack's: dropping the table makes the optimum cheaper to compute and the chosen items harder to recover.

References

- Cormen, T. H., Leiserson, C. E., Rivest, R. L., & Stein, C. (2022). *Introduction to Algorithms* (4th ed.). MIT Press. §14.2 Matrix-Chain Multiplication, pp. 370-378; §23.2 The Floyd-Warshall algorithm.
- Amakobe, M. (2025). *Advanced Computational Algorithms*, Ch. 6, §6.5, §6.7, §6.8.
- Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics*, 10(1), 196-210.
- Hirschberg, D. S. (1975). A linear space algorithm for computing maximal common subsequences. *Communications of the ACM*, 18(6), 341-343.
- Floyd, R. W. (1962). Algorithm 97: Shortest path. *Communications of the ACM*, 5(6), 345.

AI use

Claude (Claude Code) drafted the code, tests and this report's first draft from a specification I wrote from the instructions and required reading, including the hand-checked answers the tests assert. Every measurement comes from running the benchmark on my machine, and every figure is recomputed from the CSVs rather than transcribed. Full disclosure: [docs/AI_USE.md](#) .